NAME: M.Bhanu Venkata Kranthi

REG NO: 1925260268

SLOT D

COMPLETE THE TASKS WITH SUITABLE ANSWERS

TASK 1 — Understand the Problem and Identify the Data (4 Marks)

Bloom's Level: K4 — Analyse

1. Inputs, outputs and their data types

Inputs: farmer_id (string/text, because it may contain letters or leading zeros); litres (float, because milk quantity can be fractional); fat_percent (float); snf_percent (float); and shift (string or an enumerated value such as Morning/Evening). These are entered for each farmer's milk delivery.

Outputs: rate_per_litre (currency amount per litre), payment_amount (litres × rate_per_litre), and ten_day_total (sum of the farmer's payment amounts across ten days). Currency values should be represented accurately, preferably as integer paise or Decimal rather than binary float.

2. Values that should be named constants

Rules and rates that remain fixed during one program run should be constants, for example BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE, MIN_FAT_PERCENT and MAX_FAT_PERCENT. Their exact numeric values must come from the payment scheme; they should not be guessed. Naming them once avoids repeated "magic numbers" and makes an approved rate change easier and safer.

3. Validation rules

• litres must be numeric and greater than or equal to zero; a negative delivery is invalid.
• fat percentage must be within the stated range of 2.0% to 7.0%, inclusive.
• SNF percentage must be numeric and within the limits specified by the scheme.
• farmer_id must not be blank, and shift must be one of the allowed values.
• all calculated rates and payments must be non-negative. Reject invalid input with a clear message rather than silently calculating a payment.

4. Assumptions

Assume that the rate table/formula is supplied by the dairy and is authoritative; the program does not invent missing rates. Assume each record represents one farmer's delivery for one shift/day, and the ten-day total adds valid daily payments for the same farmer. The treatment of duplicate records, missing readings and rounding must be defined by the dairy before implementation.

5. Alternative design and complexity analysis

For 300 farmers, using three separate variables per farmer (litres1, fat1, snf1, and so on) creates hundreds of individual names and requires manually changing the code when the number of farmers changes. A list of records (for example, a list of dictionaries or tuples) stores each farmer's fields together and can grow to any number of entries. Processing n records takes O(n) time; storing n records takes O(n) space. Both designs can use O(n) storage, but the collection-based design scales better because it is maintainable, iterable and does not require new variable names for every farmer.

Evidence expected: a clear analysis of inputs, outputs, constants, validation rules, assumptions and why a collection-based design is preferable.

Inputs and Outputs

Inputs: farmer ID, litres, fat percentage, SNF percentage and shift for each delivery. Outputs: rate per litre, payment amount and ten-day total.

Constants

Use named constants for base rate, fat/SNF step values, permitted fat bounds, step sizes and ten-day period. Exact rate values must come from the approved dairy rate card.

Validation Rules

Farmer ID must not be blank; litres must be numeric and non-negative; fat must be within 2.0%–7.0%; SNF must be numeric and within the scheme's permitted range; shift must be allowed; payments must be non-negative.

Assumptions

Each entry is one farmer's delivery for one shift/day. Each farmer has a unique ID. The dairy supplies the official rate formula. Currency is rounded to two decimal places and the ten-day total sums valid payments.

Alternative Concept

Separate variables for each farmer are difficult to maintain. A list of dictionaries, tuples or records keeps each delivery together and can be processed with a loop.

Complexity Analysis

Processing n deliveries takes O(n) time. Storing n delivery records takes O(n) space. A running total can reduce extra storage to O(1) if individual records do not need to be retained.

Conclusion

A record-based design with named constants, clear validation and a defined rate formula is easier to test, maintain and scale to hundreds of farmers.

TASK 2 — Explore Values and Types in the Interpreter (4 Marks)

Bloom's Level: K4 — Analyse

1. Explore Python types with type()

```
>>> type(10)       # int
<class 'int'>
>>> type(10.0)     # float
<class 'float'>
>>> type("10")     # str
<class 'str'>
>>> type(True)     # bool
<class 'bool'>
>>> type([10, 8.5]) # list
<class 'list'>
```

Explanation: 10 is a whole-number integer; 10.0 is a floating-point number; "10" is text; True is a Boolean; and [10, 8.5] is a list containing numeric values.

2. Division, floor division and modulo

```
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
```

```
>>> -7 // 2
-4
```

The / operator performs true division. The // operator takes the floor (rounds down toward negative infinity), so -7 // 2 is -4, not -3. The % operator returns the remainder; 7 % 2 is 1.

3. Floating-point comparison

```
>>> 0.1 + 0.2 == 0.3
False
```

Binary floating-point cannot represent many decimal fractions exactly. Therefore, 0.1 + 0.2 is stored as a value very close to 0.3, rather than exactly equal to it. For approximate scientific comparisons, use math.isclose(); for money, use Decimal or integer paise.

4. Complete TODO 1 — use Decimal for accurate money

```
from decimal import Decimal

# 1,000 payments of Rs 33.40 and three adjustments of Rs 0.10
payments = [Decimal("33.40")] * 1000 + [Decimal("0.10")] * 3

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += amount
    return total

print("Float:", 33.40 * 1000 + 0.10 * 3)
print("Decimal:", total_decimal(payments))
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected output (the float line may show a tiny representation error):
Float: 33400.300000000006
Decimal: 33400.30
True

5. Why Decimal is appropriate for money

Decimal values created from strings preserve decimal currency amounts and avoid the binary floating-point surprise shown above. Integer paise is another reliable option: Rs 33.40 becomes 3340 paise, calculations use integers, and the result is converted back to rupees for display. round() can be useful for presentation, but rounding binary float values after each addition does not provide the same guarantee as storing money in an exact representation.

Evidence expected: interpreter prompts and outputs, explanations of each result, and a completed program showing an accurate total.

Inputs and Outputs

Inputs are Python literals/expressions such as 10, 10.0, '10', True, lists, and arithmetic expressions. Outputs are their types, evaluated results and the printed float/Decimal totals.

Constants

No application-specific constants are required. The example payment count (1,000), payment amount (Rs 33.40) and adjustment (Rs 0.10) are fixed test data and should be named if reused.

Validation Rules

Use valid Python syntax and valid numeric strings when converting input. Handle ValueError for invalid conversions. Do not use direct binary-float equality for values that require exact decimal money arithmetic.

Assumptions

The examples are run in Python 3. Decimal amounts are constructed from strings so they represent the intended decimal values exactly.

Alternative Concept

Money can be stored as Decimal or integer paise. Decimal is readable for currency; integer paise is efficient and exact when every amount is expressed in whole paise. Rounding a float after addition is less reliable.

Complexity Analysis

Each type check or arithmetic expression is O(1). Summing n payments is O(n) time; storing all n payments is O(n) space. A running sum alone uses O(1) extra space.

Conclusion

The interpreter results demonstrate Python types and division operators, while the float-versus-Decimal comparison shows why exact representations are preferred for money.

TASK 3 — Naming Rules, Constants and Dynamic Typing

1. Naming rules and conventions

Variable names should be descriptive, start with a letter or underscore, and contain only letters, digits and underscores.

Use lowercase_with_underscores for variables and functions, for example litres, fat_percent and rate_per_litre().

Use UPPER_CASE for constants, for example BASE_RATE, FAT_STEP_VALUE and SNF_STEP_VALUE.

Python does not enforce true constants. Upper-case names communicate that the values should not normally be changed.

2. Dynamic typing

Python variables refer to objects and do not have a permanently fixed type. The same variable can refer to an integer and later to a string:

```
value = 10
value = "ten"
```

After the second assignment, type(value) is str. This flexibility is useful, but input should be validated before calculations.

3. Multiple assignment and swapping

a, b = b, a

Python evaluates the right-hand side first and then assigns both values, so the values are swapped without a temporary variable.

Alternative concept and complexity analysis

If rates are repeated as magic numbers, a rate revision requires editing each occurrence and may leave inconsistent values. Central constants require changing each rate in one place. A RATE_CARD dictionary can group related rates, but is most useful when multiple rate cards must be selected. These choices do not change the calculator's O(1) calculation time per entry or its O(n) processing time for n entries; memory differences are negligible for a small fixed set of rates.

Evidence expected: one well-named set of constants and a clear explanation of naming and dynamic typing.

Inputs and Outputs

Inputs include variable names, assigned objects and pairs of values to swap. Outputs include valid assignments, the swapped values and the values returned by type() after reassignment.

Constants

Use UPPER_CASE names for values intended to remain fixed, such as BASE_RATE, FAT_STEP_VALUE and SNF_STEP_VALUE. Python treats these as a convention rather than enforcing immutability.

Validation Rules

Names must begin with a letter or underscore and may contain letters, digits and underscores. Names cannot be Python keywords. Validate user-provided values separately because dynamic typing does not guarantee suitable input.

Assumptions

Examples run in Python 3. Naming conventions are followed consistently, and constants are not reassigned during normal program operation.

Alternative Concept

Repeated magic numbers can be replaced by named constants; a dictionary can group a selectable rate card. Tuple assignment swaps values without a temporary variable.

Complexity Analysis

Assignment, lookup and swapping are O(1). Defining and using a fixed number of constants uses O(1) space. Processing n records still takes O(n) time.

Conclusion

Descriptive names, upper-case constants and an understanding of dynamic typing improve readability and reduce maintenance errors.

TASK 4 — Write Correct Expressions (4 Marks)

Bloom's Level: K4 — Analyse

1. Expressions for the milk-rate calculation

```
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
amount = round(litres * rate, 2)
```

The fat and SNF step counts are measured relative to the standard values of 3.5% fat and 8.5% SNF. Positive steps increase the rate and negative steps decrease it.

2. Operator precedence and associativity

Parentheses are evaluated first, followed by exponentiation (**), unary operators, multiplication/division, addition/subtraction, and comparisons.

Exponentiation is right-associative: 2 ** 3 ** 2 means 2 ** (3 ** 2). Unary minus must be considered carefully: -2 ** 2 evaluates to -(2 ** 2), which is -4.

Use parentheses to make the intended calculation clear and prevent precedence mistakes.

3. Floating-point step calculation

Binary floating-point cannot represent many decimal fractions exactly. Therefore (4.2 - 3.5) / 0.1 may produce 7.000000000000001 instead of exactly 7. Applying round() to the calculated step count gives the intended whole number for ordinary values.

4. round(), int() and math.floor()

round(x) rounds to the nearest integer; exact halfway cases use Python's ties-to-even rule.

int(x) truncates toward zero, so int(-2.8) is -2.

math.floor(x) returns the greatest integer less than or equal to x, so floor(-2.8) is -3.

5. Comparison chaining and Boolean operators

```
2.0 <= fat <= 7.0
fat_ok = 2.0 <= fat <= 7.0
valid = fat_ok and snf >= 0
```

Comparison chaining checks both bounds. Use and when all conditions must be true, or when at least one condition may be true, and not to reverse a condition.

Inputs and Outputs

Inputs are litres, fat, SNF, the rate constants and expressions. Outputs are the calculated fat/SNF step counts, rate, payment and Boolean validation results.

Constants

Define BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE, FAT_STEP_SIZE and SNF_STEP_SIZE as named constants supplied by the approved rate scheme.

Validation Rules

Check numeric inputs before arithmetic; require litres ≥ 0 and fat between 2.0% and 7.0%. Check SNF against the permitted scheme range. Use parentheses to make intended operator precedence explicit.

Assumptions

The supplied formula uses standard values of 3.5% fat and 8.5% SNF and step size 0.1. The exact rounding rule for step counts must be confirmed with the dairy.

Alternative Concept

Instead of floating-point step arithmetic, values expressed in hundredths can be converted to integer units before calculating steps. This can reduce representation surprises if the input precision and rounding policy are clearly defined.

Complexity Analysis

Each expression and validation check is O(1) time and O(1) extra space per delivery. Applying the calculation to n deliveries is O(n) time.

Conclusion

Correct precedence, explicit bounds and a documented rounding rule help ensure that rate and payment calculations are consistent.

TASK 5 — Validate Input and Format Output

1. Input conversion and validation

```
def read_number(text):
    try:
        return float(text)
    except ValueError:
        raise ValueError("Enter a valid number, such as 8.5.")
```

The string "8.5" converts successfully. "eight" raises ValueError. "8,5" is not accepted by float() under the usual Python locale-independent syntax; the user should enter 8.5.

2. Validation rules

Litres must be numeric and non-negative.

Fat must be within the permitted range of 2.0% to 7.0%.

SNF must be numeric, non-negative and within any system-defined permitted range.

Invalid text and values outside the permitted range should produce a clear error message rather than an incorrect payment.

3. Aligned output with f-strings

print(f"{litres:8.2f} L Fat: {fat:5.2f}% SNF: {snf:5.2f}% Amount: Rs {amount:8.2f}")

The width and precision fields align values in columns and show two digits after the decimal point.

Alternative concept and complexity analysis

Using float(text) inside try/except is concise and reliably catches invalid numeric input. Checking string patterns first can provide more tailored messages but adds code and may still miss valid formats. Both methods take O(1) time per input string for typical bounded inputs; processing n entries is O(n).

Inputs and Outputs

Inputs are strings entered for litres, fat and SNF, plus the values used for formatted display. Outputs are converted numeric values, validation messages and aligned values such as rate and amount.

Constants

Use constants for minimum/maximum fat, permitted SNF bounds and any fixed output precision such as two currency decimal places.

Validation Rules

Catch ValueError when converting text to a number. Reject negative litres, fat outside 2.0%–7.0%, invalid SNF and non-finite numeric values if they are possible. Display a clear error rather than calculating an invalid payment.

Assumptions

Users enter decimal values using a period, such as 8.5. The displayed currency amount is rounded to two decimal places.

Alternative Concept

A try/except conversion function is concise. A separate validation function provides clearer reusable rules and can return specific messages for each invalid field.

Complexity Analysis

Converting and validating one bounded input is O(1) in the intended classroom setting. Validating n records is O(n); storing the records takes O(n) space.

Conclusion

Input conversion, validation and consistent f-string formatting make the calculator safer and its results easier to read.

TASK 6 — Organise the Logic into Functions (4 Marks)

Bloom's Level: K3 — Apply

1. Suggested functions

rate_per_litre(fat, snf): accepts fat and SNF percentages and returns the rate rounded to two decimals.

payout(litres, fat, snf): accepts quantity and percentages, calls rate_per_litre(), and returns the payment rounded to two decimals.

ten_day_total(payments): accepts a sequence of payment amounts and returns their total.

2. Return value versus print

return sends a result back to the caller so it can be reused in calculations or tests. print() only displays text and does not provide the calculated value as a reusable result.

3. Local and global names

Parameters and variables created inside a function are local. Constants such as BASE_RATE can be read inside functions, but functions should not change them. Passing changed rate-card values as parameters is clearer when several seasons are supported.

4. Docstrings

```
def payout(litres, fat, snf):
    """Return the farmer payment rounded to two decimal places."""
    rate = rate_per_litre(fat, snf)
    return round(litres * rate, 2)
```

Alternative concept and complexity analysis

A single long script can work for a small program but becomes harder to test and maintain. Repeated code increases the chance of errors. Lambda expressions are best for short expressions, not multi-step validation and calculations. Function-call overhead is constant and usually insignificant compared with processing n entries, so the overall time remains O(n).

Inputs and Outputs

Inputs are arguments to rate_per_litre(), payout() and ten_day_total(). Outputs are returned rate, payment and total values; print() is used only when a value must be displayed.

Constants

Rate constants remain at module level or are passed in as a rate-card argument when different schemes must be supported.

Validation Rules

Each function should validate or receive already validated values. Require non-negative litres and permitted percentages; ensure payment collections contain numeric amounts.

Assumptions

Functions have one clear responsibility and return values instead of relying on printed text or modifying global state.

Alternative Concept

A single long script is simpler initially but hard to test. Small functions are reusable; a class may be appropriate later if the program must manage many farmers and their records.

Complexity Analysis

A single rate or payout calculation is O(1). Summing n payments is O(n). Function calls add constant overhead; storing n payments takes O(n) space.

Conclusion

Breaking the calculator into small functions improves testing, reuse and readability without changing the overall O(n) time needed to process n entries.

TASK 7 — Complete the Code

1. milkrate.py

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Return the rate per litre, rounded to two decimals."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return the payment amount, rounded to two decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        print(litres, fat, snf, rate_per_litre(fat, snf), payout(litres, fat, snf))
```

2. Worked results

To import from another script, place milkrate.py in the same folder and use: from milkrate import payout. Then call payout(litres, fat, snf).

Inputs and Outputs

Inputs are litres, fat and SNF readings for each sample delivery. Outputs are the rate per litre and payment printed by the completed milkrate.py program.

Constants

BASE_RATE, FAT_STEP_VALUE and SNF_STEP_VALUE are named constants. The sample values in the demonstration list are test inputs, not universal dairy rates.

Validation Rules

Before production use, validate numeric inputs, non-negative litres, permitted fat and SNF ranges, and the rate-card values. The sample implementation should be extended to report invalid inputs clearly.

Assumptions

The example uses a base rate of Rs 30.00, fat step value Rs 0.40, SNF step value Rs 0.30 and 0.1 percentage-point steps solely as the provided exercise formula.

Alternative Concept

The code can accept records from keyboard input, a CSV file or a list of dictionaries. A list is convenient for demonstration; file input is more useful for larger sets of farmer records.

Complexity Analysis

Rate and payout calculations are O(1) per delivery. For n deliveries, the loop takes O(n) time. The demonstration list uses O(n) space.

Conclusion

A small, reusable module with named constants is easy to import and test. Validation and an authoritative rate card should be added before real use.

TASK 8 — Algorithm, Pseudocode, Flowchart and Hand Trace

1. Pseudocode

```
CONSTANT BASE_RATE = 30.00
CONSTANT FAT_STEP_VALUE = 0.40
CONSTANT SNF_STEP_VALUE = 0.30

BEGIN
  INPUT litres, fat, snf
  IF litres < 0 OR fat < 2.0 OR fat > 7.0 OR snf < 0 THEN
      DISPLAY "Invalid input"
  ELSE
      fat_steps <- ROUND((fat - 3.5) / 0.1)
      snf_steps <- ROUND((snf - 8.5) / 0.1)
      rate <- ROUND(BASE_RATE + fat_steps*FAT_STEP_VALUE + snf_steps*SNF_STEP_VALUE, 2)
      amount <- ROUND(litres * rate, 2)
      DISPLAY rate, amount
  END IF
END
```

2. Flowchart steps

Start → Input litres, fat and SNF → Validate inputs → If invalid, display error → otherwise calculate fat and SNF steps → calculate rate → calculate amount → display result → End.

3. Hand trace

For the second and third entries, the calculations follow the rate formula shown in the code. (The expected step count for a 4.2% fat reading is 7 and for 8.7% SNF is 2.)

Alternative concept and complexity analysis

A running total uses O(1) extra space and O(n) time. Storing every entry first uses O(n) space and O(n) time. Store entries when they must be reviewed, corrected, sorted, exported or audited; otherwise a running total is more memory-efficient.

Inputs and Outputs

Inputs are litres, fat and SNF values and the algorithm’s constants. Outputs are either an invalid-input message or the calculated rate and payment; a hand trace shows intermediate steps.

Constants

The pseudocode uses the base rate, fat/SNF step values, standard percentages and step sizes as constants.

Validation Rules
Check all input bounds before calculating. If any required input is invalid, display an error and do not compute a payment. Keep pseudocode, flowchart and implementation conditions consistent.

Assumptions
The exercise formula uses the stated standard values and 0.1 step sizes. Rounding of step counts follows the rule selected for the implementation.

Alternative Concept
A flowchart emphasizes decision paths, while pseudocode emphasizes sequence and logic. A hand-trace table is useful for checking intermediate values before coding.

Complexity Analysis
One delivery requires O(1) time and O(1) extra space. Processing n deliveries requires O(n) time; a running total requires O(1) additional space.

Conclusion
Pseudocode, a flowchart and a hand trace make the algorithm easier to verify before implementation and help reveal validation or calculation mistakes.

TASK 9 — Implement and Test the Calculator (3 Marks)

Bloom's Level: K3 — Apply

1. Test cases

2. Automated testing

```
assert rate_per_litre(3.5, 8.5) == 30.00
assert payout(10, 3.5, 8.5) == 300.00
assert payout(0, 3.5, 8.5) == 0.00
```

Invalid-input tests should verify that the program raises or reports the expected validation error. A five-entry total should be calculated both by the program and independently by hand, then compared.

Alternative concept and complexity analysis
Manual testing is useful for quick exploration but can be inconsistent and difficult to repeat. A test function with assert statements repeats the same checks reliably and catches regressions after code changes. Tests take O(n) time for n test cases; their storage depends on how test inputs and outputs are retained.

Inputs and Outputs
Inputs are known valid sample cases, boundary cases and invalid values. Outputs are expected rates/payments, test pass/fail results and any validation errors.

Constants
Tests use the same approved rate constants and permitted input bounds as the implementation. Keep expected values explicit in test cases.

Validation Rules
Include tests for zero litres, minimum and maximum permitted fat, invalid text, negative litres, out-of-range fat, invalid SNF and incorrect totals.

Assumptions
Expected outputs are calculated independently from the implementation. The rate formula and rounding policy have been confirmed before judging results.

Alternative Concept
Manual testing is useful for exploration; automated assert tests are repeatable. A test framework such as unittest or pytest is more maintainable as the test suite grows.

Complexity Analysis
Running m independent tests takes O(m) time when each test performs constant work. Test data storage is O(m) if all cases are stored.

Conclusion
Boundary, normal and invalid-input tests provide stronger evidence of correctness than a single successful example.

TASK 10 — Review the Design and Recommend Improvements

1. Recommended design
Keep rate values as named constants rather than repeating magic numbers.

Keep rate calculation, payout calculation, validation and ten-day total in separate functions.

Use Decimal for financial totals when exact decimal arithmetic is required.

Use clear validation messages and formatted output.

Maintain a repeatable test set for standard, boundary and invalid inputs.

2. Limitations
The original design uses a single fixed rate card and rounds step counts to whole 0.1 percentage-point increments. The rounding rule should be confirmed with the milk union. A small local program also needs suitable data backup and validation before being used at district-union scale.

3. Evidence-based recommendation
Use constants and functions for the current fixed-rate task, and use automated tests before deployment. If rate cards change frequently or vary by season, load a rate card from configuration or pass the selected rate card into the calculation functions. Measure performance with representative data rather than assuming a design is faster.

Inputs and Outputs
Inputs are the current design, its rate scheme, records, and proposed improvements. Outputs are a design recommendation and a list of limitations and tests needed before deployment.

Constants
Keep fixed policy values in named constants or a validated rate-card configuration instead of repeating magic numbers.

Validation Rules
Validate both user data and loaded configuration. Confirm rate limits, rounding policy, duplicate-entry handling and backup requirements with the dairy.

Assumptions
The current exercise uses one fixed rate card unless otherwise specified. Performance should be measured with realistic data rather than assumed.

Alternative Concept

A fixed set of constants is simplest for one rate card. A configuration file or rate-card object is preferable when rates vary by season or location.

Complexity Analysis

Separating functions and constants does not change O(1) calculation time per delivery or O(n) processing time for n deliveries. Storing all records requires O(n) space.

Conclusion

Retain the simple modular design, add automated tests and stronger validation, and move changing policy values into configuration when requirements justify it.

TASK 11 — Read the Rate Card from a File

1. Alternative design

Store the rate card in a small JSON file and load it when the program starts. This separates policy values from calculation code and makes rate revisions easier without editing the calculation functions.

```
import json

with open("rate_card.json", "r", encoding="utf-8") as file:
    rate_card = json.load(file)

BASE_RATE = float(rate_card["base_rate"])
FAT_STEP_VALUE = float(rate_card["fat_step_value"])
SNF_STEP_VALUE = float(rate_card["snf_step_value"])
```

Example rate_card.json:

```
{
  "base_rate": 30.00,
  "fat_step_value": 0.40,
  "snf_step_value": 0.30
}
```

2. Complexity analysis

Reading a file of size t takes O(t) time and O(t) space if the full file is loaded. For a tiny rate card, the start-up cost is small. The benefit is that authorized rate changes can be made in configuration; the trade-off is that the program must handle missing, malformed or invalid configuration safely.

Inputs and Outputs

Input is a JSON rate-card file containing the base rate and step values. Outputs are the loaded configuration and calculated rate/payment using those values.

Constants

The JSON keys hold the configurable base rate and fat/SNF step values. File-path and allowed value ranges may also be constants.

Validation Rules

Handle a missing file, malformed JSON, missing keys, wrong data types and negative or unreasonable rate values. Do not start calculations with an invalid rate card.

Assumptions

The file is trusted only after validation, uses UTF-8 encoding and follows the agreed schema. Rates in the file are approved by the responsible organization.

Alternative Concept

Hard-coded constants are straightforward for a fixed exercise. JSON configuration separates policy from code; a database is more suitable when many rate cards require auditing or managed updates.

Complexity Analysis

Reading and parsing a file of size t takes O(t) time and typically O(t) memory. After loading, each rate calculation remains O(1).

Conclusion

A validated configuration file makes rate changes easier without editing calculation logic, but robust error handling and change control are essential.

TASK 12 — Re-evaluate and Adapt the Design (8 Marks)

Bloom's Level: K5 — Evaluate

Changed requirement

The milk analyser supplies fat and SNF to two decimal places. From April to June, the base rate is Rs 32.00 and each fat step is worth Rs 0.45. Farmers supplying more than 500 litres in ten days receive a bonus of Rs 0.50 per litre.

A. Analysis of changes

The input precision has changed, so step calculation must use the intended step size and a documented rounding rule.

The base rate and fat-step value now depend on the season; a single fixed constant set is no longer sufficient.

A bonus depends on the ten-day total quantity and must be applied only when litres exceed 500.

The functions and tests must be updated to cover seasonal rate cards and the bonus threshold.

B. Evaluation of original design

The original constants and functions are suitable for one fixed rate card, but they do not directly represent seasonal rates or a ten-day volume bonus. The modular functions can be retained with parameters for the selected rate card and a separate bonus calculation.

C. Adapted implementation

```
STANDARD_CARD = {"base": 30.00, "fat_step": 0.40, "snf_step": 0.30}
SEASONAL_CARD = {"base": 32.00, "fat_step": 0.45, "snf_step": 0.30}
FAT_STEP = 0.1
SNF_STEP = 0.1
BONUS_THRESHOLD = 500.0
BONUS_PER_LITRE = 0.50

def select_rate_card(month):
    """Return the rate card for a month (1–12)."""
    return SEASONAL_CARD if 4 <= month <= 6 else STANDARD_CARD

def rate_per_litre(fat, snf, card):
    """Calculate the rate using the selected rate card."""
    fat_steps = round((fat - 3.5) / FAT_STEP)
    snf_steps = round((snf - 8.5) / SNF_STEP)
    return round(card["base"] + fat_steps * card["fat_step"] + snf_steps * card["snf_step"], 2)

def ten_day_payment(litres, fat, snf, month, ten_day_litres):
    """Return payment including the applicable ten-day volume bonus."""
    card = select_rate_card(month)
```

```
    amount = litres * rate_per_litre(fat, snf, card)
    if ten_day_litres > BONUS_THRESHOLD:
        amount += litres * BONUS_PER_LITRE
    return round(amount, 2)
```

Assumption: the seasonal rate uses the same SNF step value as the original rate card because the changed requirement specifies a new base rate and fat-step value but does not specify a new SNF-step value. Confirm this assumption with the union before production use.

D. Evidence-based defence and tests

```
assert select_rate_card(4)["base"] == 32.00
assert select_rate_card(7)["base"] == 30.00
assert round(10 * (30.00), 2) == 300.00
assert round(501 * 0.50, 2) == 250.50
```

Test both a month within April–June and a month outside that period, and test ten-day quantities of exactly 500 litres and greater than 500 litres. The bonus must not apply at exactly 500 litres because the requirement says “more than 500”. The redesign is more flexible but introduces extra configuration and test requirements.

Inputs and Outputs

Inputs are month, litres, fat, SNF, ten-day total litres and the selected rate card. Outputs are the selected seasonal card and payment with any eligible volume bonus.

Constants

Define the standard and seasonal rate cards, step sizes, bonus threshold (500 litres) and bonus per litre (Rs 0.50). The seasonal SNF step value must be confirmed because the changed requirement does not specify it.

Validation Rules

Validate month from 1–12, non-negative litres, fat and SNF bounds, and valid rate-card values. Apply the bonus only when ten-day litres are strictly greater than 500; exactly 500 does not qualify.

Assumptions

April through June use the seasonal base rate Rs 32.00 and fat step Rs 0.45. The example retains the original SNF step value only as an explicit assumption pending confirmation.

Alternative Concept

Use a rate-card dictionary selected by month, or pass a RateCard object into calculation functions. A separate bonus function keeps the threshold policy independent of the base-rate calculation.

Complexity Analysis

Selecting a card and calculating one payment are O(1). Processing n delivery records is O(n). Storing all records is O(n); computing a total as a stream can use O(1) extra space.

Conclusion

The modular design can adapt to seasonal rates and volume bonuses, but the revised policy and edge cases must be documented and tested before the program is used.